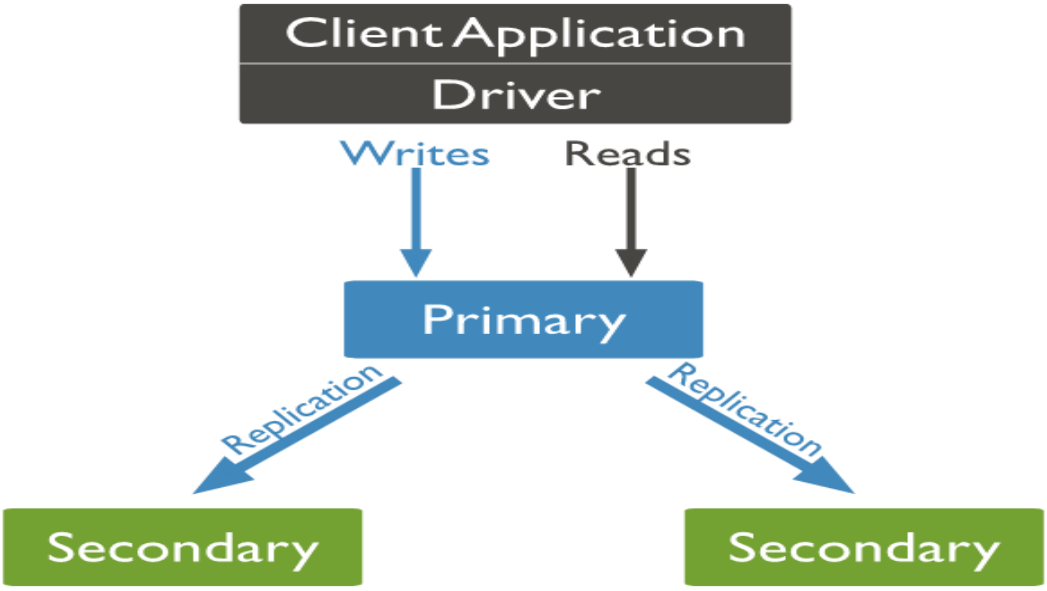
MongoDB Replication

Understanding MongoDB Replication

A replica set in MongoDB is a group of MongoDB servers that maintain the same data set. Replica sets provide redundancy and high availability, and are the basis for all production deployments.



A replica set contains several data bearing nodes and optionally one arbiter node. One of the data bearing node will assume the role of primary others are secondaries.

Replication Options in mongod.conf file

```
replication:
   oplogSizeMB: <int>
   replSetName: <string>
   enableMajorityReadConcern: <boolean>
```

Except replSetName other options are optional.

OpLog

Primary records all changes to log called as OpLog. The secondary members then copy and apply these operations asynchronously. All replica set members contain a copy of the oplog, in the `local.oplog.rs collection`, which allows them to maintain the current state of the database.

The entries in oplog produces same result whether they are applied one time or multiple time on target database.

opLog size & retention hours:

On windows and linux systems opLog can take up about 5% free disk space or the size od opLog can grow between 990MB to 50GB.

If Oplog size is not set in configuration file before replicaset is initialized then MongoDB sets to around 5% free disk space available and resizing can be done by using MongoDB admin command

```
db.adminCommand(
    {
      replSetResizeOplog: <int>,
      size: <double>,
      minRetentionHours: <double>
    }
 )

replSetResizeOplog:  should be set 1
size between 990MB to 1 Petabytes
minRetentionHours :  (hours to retain the Oplog)
```

This change has to be persisted by entering the corresponding entries in mongod.conf file

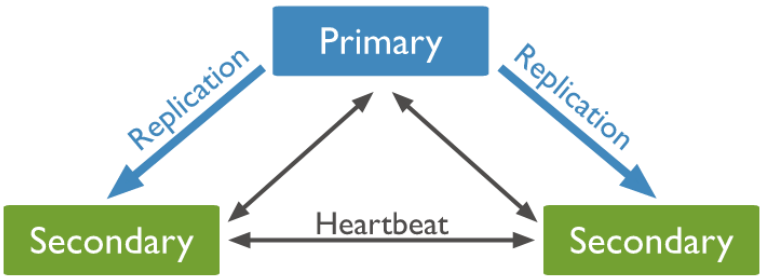
Check Oplog Status

The Following method can be used to query current Oplog status and configuration information.

```
rs.printReplicationInfo()
```

HeartBeat

Replica set members send heartbeats (pings) to each other every two seconds. If a heartbeat does



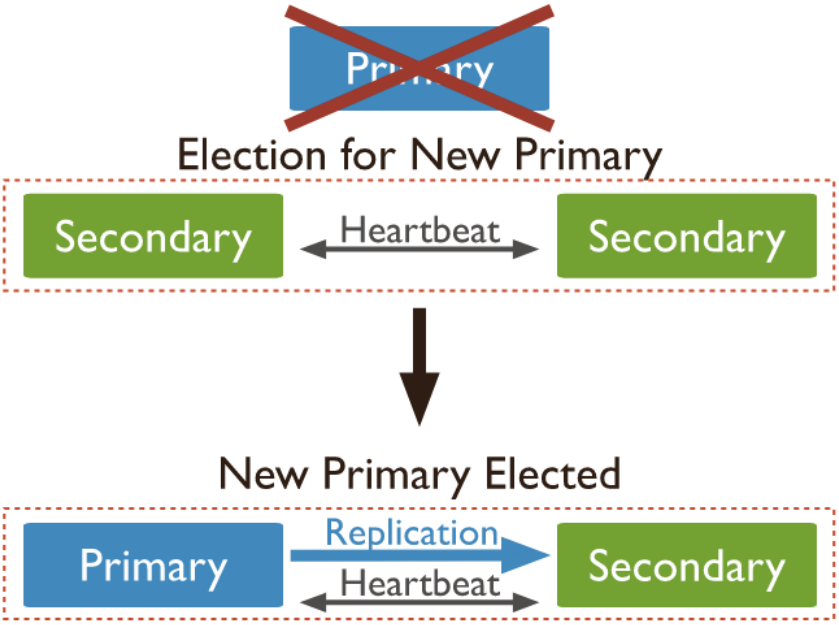
not return within 10 seconds, the other members mark the delinquent member as inaccessible.

Asynchrones Réplication

Secondaries apply operations from the primary asynchronously.

Automatic Failover

When a primary does not communicate with the other members of the set for more than the configured electionTimeoutMillis period (10 seconds by default), an eligible secondary calls for an election to nominate itself as the new primary. The cluster attempts to complete the election of a new



primary and resume normal operations within 12 seconds.

During time of election process replica set can serves read queries with readPref({"secondary"}) set to secondary continue to work.

Replica Set Elections

Replica sets use elections to determine which set member will become primary. Replica sets can trigger an election in response to events, such as:

- Initiating a replica set
- Adding a new node to the replica set
- Performing replica set maintenance using methods such as rs.stepDown() or rs.reconfig()
- Primary loosing connectivity with majority of secondaries
- Secondary members losing connectivity to the primary

Types of Nodes

- Primary Node

- Secondary Node
    - Data Bearing Node
    - Hidden Node
    - Delayed Node
- Arbiter Node

Primary Node

Primary node accepts all read and write operation in replica set and no other node is allowed to perform write operations. Primary node is elected by replica set after an election process. and will remain primary until another election is called.

Secondary Node

Secondary nodes keeps a copy of all data from primary node. As changes to data occur on primary node secondary node gets a copy of them applies locally thus keeping themselves unto date. In event of primary failure a secondary node is elected as new primary by replica set.

Hidden Node

A secondary node can be configured as hidden node where specific read operations can be performed like Reporting, Data mining etc. Hidden nodes cannot become primary node. The priority of hidden node must be set to 0.

Delayed Node

A secondary node can be configured as delayed node. A delayed node applies the updates from the primary after the delay time thus always running behind the primary (whole replica set). Delay node are useful for data recoveries from accidental data deletions. A delayed node cannot become primary node. The priority of delayed node must be set to 0.

Arbiter Node

Arbiter nodes are special nodes where they dont replicate data but are only part of replica set as to vote in election during split brain scenario. A Arbiter node can vote in election and help any of the eligible secondaries be elected as primary in case of issue but cannot become primary.

Setting up Replica Sets in MongoDB

Let's setup a MongoDB replication using 3 nodes running on same server. Lets call them node1, node2 and node3

Creating the keyfile and setting permissions on it:

```
sudo mkdir -p /var/mongodb/pki/
sudo chown mongod:mongod /var/mongodb/pki/
```

Creating Key File:

```
openssl rand -base64 741 > /var/mongodb/pki/syn-keyfile
chmod 400 /var/mongodb/pki/syn-keyfile
```

Lets Create Node configuration files

First node configuration (node1.conf):

```
storage:
net:
dbPath: /var/mongodb/db/node1
bindIp: 192.168.15.59,localhost
port: 27011
security:
authorization: enabled
keyFile: /var/mongodb/pki/m103-keyfile
systemLog:
destination: file
path: /var/mongodb/db/node1/mongod.log
logAppend: true
processManagement:
  fork: true
replication:
replSetName: syn-repl
```

Creating the dbpath for node1:

Now copy the node1.conf for all nodes.

```
cp node1.conf node2.conf
cp node1.conf node3.conf
```

Editing node2.conf using vi: vi node2.conf

Saving the file and exiting vi: :wq

node2.conf, after changing the dbpath, port, and logpath:

```
storage:
net:
dbPath: /var/mongodb/db/node2
bindIp: 192.168.15.59,localhost
port: 27012
security:
keyFile: /var/mongodb/pki/m103-keyfile
systemLog:
destination: file
path: /var/mongodb/db/node2/mongod.log
logAppend: true
processManagement:
  fork: true
replication:
replSetName: syn-repl
```

node3.conf, after changing the dbpath, port, and logpath:

```
storage:
dbPath: /var/mongodb/db/node3
net:
bindIp: 192.168.15.59,localhost
port: 27013
security:
keyFile: /var/mongodb/pki/m103-keyfile
systemLog:
destination: file
path: /var/mongodb/db/node3/mongod.log
logAppend: true
processManagement:
fork: true
replication:
replSetName: syn-repl
```

Creating the data directories for node2 and node3:

```
mkdir -p /var/mongodb/db/node{1..3}
```

Start all nodes

```
mongod -f node1.conf
```

```
mongod -f node2.conf
mongod -f node3.conf
```

Connect to First Node

```
mongosh --port 27017
```

Initiating the replica set:

```
rs.initiate()
```

Creating a user:

```
use admin
db.createUser({
user: "admin",
pwd: "pass12345",
roles: [
{role: "root", db: "admin"}
]
})
```

Exiting out of the Mongo shell and connecting to the entire replica set:

```
exit
```

Now connect back to mongodb replicaset instead of individual node

```
mongo --host "syn-
repl/192.168.15.59:27011,192.168.15.60:27012,192.168.15.61:27012"   -u
"admin"  -p "pass12345" --authenticationDatabase "admin"
```

Getting replica set status:

```
rs.status()
```

Adding other members to replica set:

```
rs.add("node2:27012")
rs.add("node3:27013")
```

Getting an overview of the replica set topology:

```
rs.status()
rs.isMaster()
```

Stepping down the current primary:

```
rs.stepDown()
```

Checking replica set overview after election:

```
rs.isMaster()
```

Enabling read commands on a secondary node:

Adding or Removing Nodes

Any time a node can be added or removed from replica set. When a new node (non arbiter) is added it will perform a full data sync from the replica set and then start getting updates from the primary node.

Adding a Node

Make sure the mongodb server which you wish to add in the replica set is configured and already running. And the mongodb should not have any of its own data.

Connect to primary node and run the following command.

```
rs.add("<hostname or IP address>:<port no>")
```

The above command will add the new node and trigger data sync to new node.

Adding a Arbiter Node

Arbiter nodes does not hold any data but they help over come split brain scenarios.

Connect to primary node and run the following command.

```
rs.addArb("<hostname or IP address>:<port no>")
```

Removing a Member Node

Removing a member involves removing the membership of such node from replica set configuration and forcing all members of replica set to disconnect from it which may take some time (max 5 minutes).

Note: Removing member triggers election.

```
rs.remove(<hostname>)
```

The member mentioned will be removed from replica set.

Changing configuration on a Running Replica Set

A running replica sets member configuration can be changed using the following methods

```
rs.conf()
rs.reconfig()
```

The `rs.conf()` method returns a document with current configuration which can be examined to understand current configuration or the same can be stored in a variable to reconfiguration.

Sample of `rs.conf()`

```
{
  _id: <string>,
  version: <int>,
  protocolVersion: <number>,
  writeConcernMajorityJournalDefault: <boolean>,
  configsvr: <boolean>,
  members: [
    {
      _id: <int>,
      host: <string>,
      arbiterOnly: <boolean>,
      buildIndexes: <boolean>,
      hidden: <boolean>,
      priority: <number>,
      tags: <document>,
      slaveDelay: <int>,
      votes: <number>
    },
    ...
  ],
  settings: {
    chainingAllowed : <boolean>,
    heartbeatIntervalMillis : <int>,
    heartbeatTimeoutSecs: <int>,
    electionTimeoutMillis : <int>,
    catchUpTimeoutMillis : <int>,
    getLastErrorModes : <document>,
    getLastErrorDefaults : <document>,
    replicaSetId: <ObjectId>
  }
}
```

ReplicaSet wide Settings:

`chainingAllowed` controls whether secondaries can replicate from other secondaries instead of directly from the primary

`heartbeatIntervalMillis` controls how often members send heartbeat signals to each other.

`heartbeatTimeoutSecs` → controls how long before declaring a node unreachable (default 10s)

`catchUpTakeoverDelayMillis` → delay before another node takes over catch-up if the elected primary is stuck.

`electionTimeoutMillis` → controls election speed (default 10,000 ms)

`catchUpTimeoutMillis` in MongoDB replica sets defines how long a newly elected primary is allowed to remain in the catch-up phase before either completing or aborting.

`writeConcernMajorityJournalDefault` when set to trueMongoDB ensures the write is both replicated to a majority of nodes AND written to their journals before acknowledging for all writes with WriteMajority `{w: "majority"}`.

Node Level Settings:

`cfg.members[n].priority = n` --Number between 0 and 1000 for primary/secondary; 0 or 1 for arbiters. higher number means higher priority

`cfg.members[n].votes = n` -- determines if the member can vote or not possible values 1 or 0

`cfg.members[n].hidden = false/true` -- determines if the member is visible to clients with readPreference set to secondary

`cfg.members[n].tags = { "region": "analytics" }`

`cfg.members[n].secondaryDelaySecs` -- The number of seconds "behind" the primary that this replica set member should "lag".

Steps to Change Configuration:

Following steps can be used to change configuration

Assigning current configuration to a variable

```
cfg = rs.conf()
```

Modifying the Configuration

```
cfg.members[n].votes = 0
cfg.members[n].hidden = true
cfg.members[n].priority = 0
```

Applying the configuration

```
rs.reconfig(cfg)
```

Checking Replication Status

Replication status including replication lag info can be checked by using following method

```
db.getReplicationInfo()
```

Output

`configuredLogSizeMB:` Size of Oplog configured

`logSizeMB:` Current Oplog Size (sometime it may be larger than configured size)

`usedMB:` Current used Oplog size

`timeDiff:` Time Difference in seconds between oldest transaction and newest transaction from Oplog. (more accurate compared to hours)

`timeDiffHours:` Time Difference in Hours between oldest transaction and newest transaction from Oplog.

`tFirst:` oldest transaction timestamp

`tLast:` Newest transaction timestamp

`now:` Current datetime

`rs.printSecondaryReplicationInfo()`

Output

`syncedTo:` gives time till the Secondary synced with Primary

`replLag:` n seconds behind the primary '